

//insertAfter a node to a singly linked list

```
#include <iostream>
using namespace std;

struct node {
    int data;
    node* next;
};

class SLinkedList {
private:
    node* head;
    node* tail;

public:
    SLinkedList() {
        head = nullptr;
        tail = nullptr;
        cout << "head and tail nodes are initiated with nullptr" << endl;
    }

    void ListAppend(int elem) {
        node* newNode = new node;
        newNode->data = elem;
        newNode->next = nullptr;

        if (head == nullptr) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            tail = newNode;
        }
    }

    void ListPrepend(int elem) {
        node* newNode = new node;
        newNode->data = elem;
        newNode->next = nullptr;

        if (head == nullptr) {
            head = newNode;
            tail = newNode;
        } else {
```

```

        newNode->next = head;
        head = newNode;
    }
}

void InsertAfter(node* curNode, int elem) {
    node* newNode = new node;
    newNode->data = elem;
    newNode->next = nullptr;

    if (head == nullptr) {
        head = newNode;
        tail = newNode;
    } else if (curNode == tail) {
        tail->next = newNode;
        tail = newNode;
    } else {
        newNode->next = curNode->next;
        curNode->next = newNode;
    }
}

void ListDisplay() {
    node *tmp = head;
    while (tmp != nullptr) {
        cout << tmp->data << " ";
        tmp = tmp->next;
    }
    cout << endl;
}

node* GetNode(int value) {
    node* tmp = head;
    while (tmp != nullptr) {
        if (tmp->data == value) {
            return tmp;
        }
        tmp = tmp->next;
    }
    return nullptr;
}

};

int main() {

```

```

SLinkedList numList1;
numList1.ListAppend(30);
numList1.ListAppend(40);
numList1.ListPrepend(20);
numList1.ListPrepend(10);

numList1.ListDisplay();

node* curNode = numList1.GetNode(30);
if (curNode != nullptr) {
    numList1.InsertAfter(curNode, 35);
}

numList1.ListDisplay();

return 0;
}

```

```

head and tail nodes are initiated with nullptr
10 20 30 40
10 20 30 35 40

```

//Singly linked list code with $O(1)$ functions under a class

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

class SinglyLinkedList {
private:
    Node* head;
    Node* tail;
    int listSize;

public:
    SinglyLinkedList() : head(nullptr), tail(nullptr), listSize(0) {}
    bool empty() {
        return listSize == 0;
    }

```

```

}
int size() {
    return listSize;
}
int front() {
    if (head != nullptr) {
        return head->data;
    }
    throw runtime_error("List is empty!");
}
int back() {
    if (tail != nullptr) {
        return tail->data;
    }
    throw runtime_error("List is empty!");
}
void push_front(int newData) {
    Node* newNode = new Node();
    newNode->data = newData;
    newNode->next = head;
    head = newNode;
    if (tail == nullptr) {
        tail = newNode;
    }
    listSize++;
}

void push_back(int newData) {
    Node* newNode = new Node();
    newNode->data = newData;
    newNode->next = nullptr;

    if (tail == nullptr) {
        head = newNode;
        tail = newNode;
    } else {
        tail->next = newNode;
        tail = newNode;
    }
    listSize++;
}

void pop_front() {
    if (head == nullptr) {
        throw runtime_error("Cannot pop from an empty list!");
    }
}

```

```

    }

    Node* temp = head;
    head = head->next;
    delete temp;
    listSize--;

    if (head == nullptr) {
        tail = nullptr;
    }
}

void printList() {
    if (empty()) {
        cout << "List is empty." << endl;
        return;
    }
    Node* temp = head;
    while (temp != nullptr) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "nullptr" << endl;
}

~SinglyLinkedList() {
    while (head != nullptr) {
        Node* temp = head;
        head = head->next;
        delete temp;
    }
}

};

int main() {
    SinglyLinkedList list;

    list.push_back(10);
    list.push_front(20);
    list.push_back(30);
    list.push_back(40);
    list.push_front(50);

    cout << "List after push operations: ";

```

```
list.printList();

cout << "Size of the list: " << list.size() << endl;
cout << "Is the list empty? " << (list.empty() ? "Yes" : "No") << endl;

cout << "Front of the list: " << list.front() << endl;
cout << "Back of the list: " << list.back() << endl;

list.pop_front();
cout << "List after pop_front: ";
list.printList();

return 0;
}
```

```
List after push operations: 50 -> 20 -> 10 -> 30 -> 40 -> nullptr
Size of the list: 5
Is the list empty? No
Front of the list: 50
Back of the list: 40
List after pop_front: 20 -> 10 -> 30 -> 40 -> nullptr
```